

Abstract

Project: Programming Language C++, Evolution Working Group

Matt Calabrese (metaprogrammingtheworld@gmail.com)

2015-09-25

This paper evaluates the current specification of `void` in C++ as an incomplete type with exceptional rules and proposes changes that would update `void` to instead be a complete, object type. The intent of this modification is to eliminate special casing in generic code where `void` may appear as a dependent type, such as the existing `std::promise<void>` specialization in the standard library, and to do so in a way that minimizes breakage of existing code. This also has the positive side-effect of simplifying the standard as a whole, due to the amendment or outright removal of several clauses that currently need to treat `void` separately and explicitly.

Motivation

One of the many powerful aspects of C++ is its ability to represent generic functions and types as exemplified by the algorithms and containers of the standard library, which are able to operate on built-in and user-defined types equivalently provided that the types involved model the necessary concepts. One of the underlying properties of the language that makes this feasible is the potential for user-defined types to behave in both a syntactically and semantically similar manner to built-in types. This is because the language gives built-in object types and user-defined types value semantics, and allows for overloading and generic functions by way of templates. In practice this usually works well, however, `void` types often pose difficulty in generic code because they are incomplete and not instantiable, even though they can appear as the result type of a function that may be invoked in a generic setting. They are very unique with respect to built-in types, let alone user-defined ones.

Consider the following example, based on real-world code:

```
// Invoke a callable, logging its arguments and return value.
template<class R, class... P, class Callable>
R invoke_and_log(callable_log<R(P...)>& log, Callable&& callable, P... args) {
    log.log_arguments(args...);
    auto result = std::invoke(std::forward<Callable>(callable),
                             std::forward<P>(args)...);
    log.log_result(result);
    return result;
}
```

The above code is simple enough, however, it breaks down in one, very important case -- when the return type of the `Callable` happens to be `void` (there is another class of cases not handled here, though this is to keep the example simple). In the `void` case, we'd still expect `callable` to be invoked and to have its arguments logged. Perhaps more subtle, the call to `log.log_result(result)` is meaningful even if serialization of a `void` were to end up being a no-op, since the invocation of the function still signals a successful execution of `callable` (i.e. if we get there, `std::invoke` did not propagate an exception). Ideally, all of this code would just work as written and without further specialization or indirection to account for the case where `callable` returns `void`.

Example in the Standard

The motivation for this change does not exist solely in user-written generic code, as symptoms of the underlying problem are visible even in the standard library itself. Most clearly, this can be seen with the need for specializations of `std::promise` and `std::future` for the `void` type. It can further be seen in proposals such as the `void` specialization for the expected template². Unfortunately, even though we already have specializations for `std::promise` that at least allow `std::promise<void>` to be created, dealing with that type in generic code by users is still difficult due to the way that `void` is handled in the language and the fact that the specializations provide no common interface for setting the promised value (this latter point does not actually have to be the case even without changes to the language itself, though such an interface would be less than ideal). An example of where things break down in simple generic code that deals with `std::promise` can be seen here:

```
template<class T, class Fun>
void set_value_with_result(std::promise<T>& p, Fun fun) {
    p.set_value(fun());
}
```

In the above code, if `T` happens to be `void`, our code will fail because `set_value` in that case takes no parameters other than the `promise` itself, and you also cannot pass a `void` expression as a function argument.

Note: This paper strictly addresses language changes to `void` without proposing significant changes to the standard library. If the language changes are considered viable, a separate proposal is planned that would specify updates to the standard library, such as the removal of the `void` specializations of `std::promise` and `std::future` in a way that would be relatively seamless for existing users.

Problem Description

Generic programming in C++ is especially powerful because with care, concept requirements can usually be specified in a way such that they may be met nonintrusively with respect to an existing type, making it possible to have the type model a concept even if a programmer otherwise has no control over its implementation (i.e. it is a part of a dependency or is a built-in type). However, sometimes requirements are specified that simply cannot be met nonintrusively. A good example of this is when a type requirement is the instantiability of that type, or proper semantics regarding special member functions. In these cases, if the type does not already meet these requirements, they cannot be met without modifying the type in question. If the type in question is a part of a dependency or happens to be a fundamental type of the language, then that type cannot be made to model the given concept without either patching the dependency or changing the language, respectively. This is particularly unfortunate if a type *could* model the concept without any cost, yet simply does not do so.

The problem that generic libraries face when dealing with `void` can be thought of as precisely this last case. As currently specified in the language, `void` cannot be instantiated, nor can it be copied or compared, even though it can be logically thought of as a monostate type.

Recommended Solution

Briefly, the solution that is recommended by this proposal is to change `void` to a complete object type and to do so in a way that prevents considerable breakage to existing code. This also implies that `void` can appear as a function parameter, that you can form references to `void`, and that you can do anything else that is expected of other object types. Although this change may sound risky, the alterations are specified in such a way that potential breakage is limited. Descriptions of potential breaking cases are detailed later.

In reality, it is more likely that this change could be made in such a way that most existing users of the language would not even notice the update, and their code would not be affected adversely by such ignorance. What is proposed more formally below is a fairly straight-forward simplification to the standard, removing many parts that currently need to explicitly mention `void` types in wording due to their nature as a unique kind of type in C++. The most risky of changes are discussed in this document explicitly, separate from the actual changes that are suggested for the standard.

Considerations

Unary Functions Taking Void

As `void` becomes a Regular¹ type, it is essential that it is allowed as function parameter types, just like any other object type in the language, otherwise it is not suitable in generic code, as seen in the logging and `std::promise` examples. This, however, brings about one minor, though unfortunate problem. Syntactically, in current C++, a type such as `int(void)` is valid yet is not a function taking `void` as a parameter, but rather, it is a function taking no parameters. In other words, even though it is syntactically different, it specifies the same type as `int()`. This is, perhaps, obvious, but it means that we would not be able to reclaim that syntax without causing a large amount of breaks to existing code. What's worse is that sometimes those breaks could hypothetically cause compilation or linking to still succeed, but have different results.

The suggested solution to this problem is to not reclaim this syntax, at least not at this time, when the `void` type is not dependent. Whether or not the syntax should be deprecated for future reclamation is questionable, though it is recommended in this proposal that we do so. Importantly, we can and should use this syntax when `void` is dependent, as is explicitly suggested in this proposal, and is thankfully not allowed in the present state of the language. Because it is not allowed currently, it is less likely (though still possible in hypothetical cases) that such a change would break existing code. The reason why it is important that we allow this syntax in a dependent context is because this is precisely where it matters to generic code. By utilizing the syntax in this context, we have little-to-no chance of breaking any existing code while also allowing for generic code to be written in a way that does not need to specialize or go through indirection in the event that `void` happens to show up as a dependent type.

This leaves us with one question, though. If, when in a non-dependent context, you cannot declare a function that takes a single, `void` parameter using this syntax, how exactly would you specify such a function type? There are multiple options, but ultimately it is recommended that users either specify the `void` parameter type with cv-qualification, which is not valid in this context in current C++, or alternatively they can simply give the parameter a name, which is also not currently allowed. Both of these solutions are specified as a part of this proposal. Rationale for why this seemingly subtle syntax is the preferred solution is presented later.

Finally, it deserves to be repeated that this quirkiness is only required when the function in question takes exactly one parameter, where that parameter is `void`, and where that `void` is not a dependent type. If the function type takes multiple parameters, then the user needs to do nothing special. Similarly, if the `void` type is dependent, the user also needs to do nothing special. This only would affect a small amount of yet-to-be-written code that only a certain class of users would ever want to write, and it does not at all negatively affect the simplicity of writing generic code.

Linking Breakages

Because we are changing the `void` type, it might initially seem as though we'd, in practice, be breaking compatibility with existing, prebuilt libraries, including C libraries, and thereby force them to no longer link correctly. This is actually untrue. Since our alterations do not add any state to the `void` type, no additional or different information needs to be communicated, and in practice, implementations would be able to retain compatibility.

SFINAE

While mostly theoretical, there are potentially SFINAE exploits that might be in use that could take advantage of `void`'s properties, such as its inability to be used as a function parameter type in a dependent context, and its inability to be used in a `sizeof` expression. At the very least, such exploits are not known to have been used in standard library implementations nor have they been encouraged in Boost. It is suspected that such properties are exploited rarely, if it all, and in those few cases if and where they are exploited, there already exist drop-in alternatives via other forms of SFINAE exploits. Note that the `sizeof` trick that is or was frequently used when defining metafunction traits is distinct from what is described here and would not be affected.

Ternary Operator

While most of the changes proposed are simple, there is one somewhat non-trivial change made to the ternary operator. This should be examined more closely to be certain that there is no adverse effect by this change. An implementation is not provided along with this proposal.

Alternatives

The solution that is proposed was arrived at after carefully considering possible alternatives. Notable suggestions from other parties are presented below.

Library-Based Solutions

As described in the motivation section, the current way that generic code deals with `void` is by special-casing code, often through helper templates. It has been suggested that we instead possibly consider standardizing helper templates that ease dealing with `void` in generic code. The problems with this are that such facilities would be both complicated to specify correctly and would require people writing generic code to actually know about and use them. When and why it would be important is subtle, and we'd likely not be able to cover all cases. Readers of that code who are not familiar with the problem would also be baffled by the subtleties (a problem that already exists today when writing such generic code, which a change to `void` would remedy). By simply making `void` an object type, we solve our problems completely and in a way that is easy for users to utilize and also easier to specify fully and properly in the standard when compared to a complicated set of library facilities.

Void as *Almost* an Object

The idea here is that instead of making `void` an object type, we instead just make it act a little bit like one in certain contexts. The proclaimed rationale for this approach is that we introduce fewer changes to the meaning of `void` so as to minimize breakages and to not make `void` do more than is necessary. For instance, it was suggested that we make it possible to create an instance of `void` on the stack, but not be able to pass it to a function as an argument, or make an array or tuple that contains a `void` element. While the intention here is in the right place, the rationale is flawed in several ways:

- By just making `void` *almost* an object, we'd only be trading some special-casing in generic code for other special-casing.
- The changes to the standard that would make `void` an object are simpler to specify than trying to make `void` *almost* an object type with special rules.
- All of the functionality of a `Regular` type really is required since things like function result types are often worked with in this manner in generic code.
- There is no evidence that an intermediate step in the re-specification of `void` as an object type would at all reduce breakage to existing code when compared to making the full change. It could, however, scatter the complete changes across multiple standards, leaving the language with unnecessarily subtle differences between iterations.

An Entirely New Void Type

Also suggested was the possibility of introducing a new type to the language, separate from `void`, that behaves in the way that this proposal specifies the existing `void` type should behave. The idea of this solution is that people who want such a monostate type can opt-in for it, leaving the existing `void` type unchanged. This initially may seem like a reasonable solution, but it fails to actually solve the generic programming problem since *users* of the generic code would be the ones who would be required to now take advantage of that new type. Library developers would still need to special-case for `void` in practice because of this.

Explicit Void

As described earlier, this proposal recommends using *cv-void* or a named `void` parameter for representing a unary function with a `void` parameter when `void` is not dependent. One alternative to using *cv-void* or a named `void` parameter for representing a unary function having a `void` parameter is to instead propose an entirely new syntax for that case. For instance, one possibility is to allow a declaration such as:

```
void foo(explicit void) {}
```

At first this may seem like a good idea since it makes the difference between it and the current meaning of a `void` parameter list more drastic, however it has a few drawbacks:

- It introduces a syntax that would only work for `void` (unless it were also proposed to work for other types for the sake of consistency).
- It is a more complex change as it requires altering the language grammar.
- It introduces a syntax that would likely be deprecated in the future.

The last point is a bit subtle, though it may be the most important. Ultimately, in a future standard it would be a simplification to the language to make a parameter-list consisting of a single, unnamed, *cv-unqualified* `void` parameter behave no different from most other object types. That is to say, that syntax would eventually be used to declare a unary function taking a parameter of type `void`. It is not recommended that we update the meaning of that syntax immediately as a part of this proposal because that would change the meaning of a large body of existing code in a way that would cause breakages, including sometimes in ways that only change behavior of existing programs at runtime. By at most deprecating the current meaning of `void` in a parameter list, we'd effectively provide a buffer for existing code to be updated such that the further update can potentially take place in some future standard. If, in the meantime, we were to introduce an entirely new syntax and alter the language grammar solely to allow the declaration of a parameter of type `void`, we'd likely want to eventually deprecate and remove *that* syntax once the desired syntax is ripe to be used. On the other hand, if in the time being we only use *cv-qualified* `void` or a named `void` parameter to represent an actual `void` parameter, then if/when the current meaning of `void` is to finally be replaced, any code that declared a single parameter of type `void` in the currently proposed manner would not need to be updated, since even then it would remain as a valid way to declare such a parameter (just as it is a valid way in the language right now to declare parameters of non-void type).

So, at the cost of being a little bit subtle in newly-written code, we avoid cluttering the language with new syntax and also avoid having to remove a special syntax at some point in the future. It is a simpler change in the language to make right now, in addition to being better in the long term.

Parameter Collapsing

In discussions, it has been suggested by multiple parties independently to consider "collapsing" `void` parameters in a function type in a way that makes a function type specified as `void (int, void)` equivalent to the function type `void (int)`. Similarly, when invoking a function and passing a `void` expression as an argument, that argument would instead be ignored and the invocation would actually be passing one less argument than it would appear. Though initially this might not look problematic, not only does it keep `void` as a special type with special rules that can really make it difficult to write certain generic code, but it also introduces subtle ambiguities or other errors. For example:

```
enum class log_option { simple, verbose };

template<class T>
void log(T what_to_log, log_option option = log_option::simple);

template<class Fun>
void log_result(Fun fun) {
    log(fun(), log_option::verbose);
}

int main() {
    // logs an int using the verbose option
    log_result([]{ return 0; });

    // If internally the first argument "collapses" because the function returns void,
    // then we are no longer logging "void" with the "verbose" option, but instead,
    // we are logging an instance of "log_option" with the "simple" option.
    log_result([]{});
}
```

Size Zero Void

One suggestion that has repeatedly come up is to have `sizeof(void)` report `0` and to allow multiple instances to share the same

address. This would prevent users from having to use tricks akin to the empty base optimization in order to make more optimal usage of memory. Ideally, this would be the case, however such a change to the language is both vast and out of scope of the proposal. Allowing a type to have a `0` size and to allow separate instances to share an address implies drastic and subtle breaking changes to existing code. For instance, if you were to make an array of such a `void` type, a pointer, at least in the traditional sense, would no longer be able to be used as an iterator into that array (notably meaning that generic code which relies on this would now fail for such a size `0` type). As well, any code that relies on an object's type and address as unique would fail for `void` types, even though it is otherwise perfectly acceptable. Finally, if such a size were permitted for `void`, it should really be allowed for any type, including user-defined types. Having a special rule for `void` would make one more thing to have to think about and deal with differently for `void` types. Instead, this proposal opts to leave the size of `void` unspecified and thereby governed by existing language rules. In practice, it is expected that `void` will likely be size `1` in most implementations, though this is not required. If an eventual change were made to the language to allow for size `0` types, then `void` would be able to implicitly take advantage of that.

Proposed solution

The `void` type is updated to be an object type with care taken to minimize breaking changes to existing code. This implies a considerable number of changes to the standard, although most of those changes are simplifications due to the removal of redundant statements mentioning `void`, which are now implicitly covered by the fact that `void` is a complete object type that is considered a scalar. Additional updates are planned for the LEWG, such as updates to `std::promise`, but they are outside of the scope of this proposal and are predicated on the viability of the language changes presented here.

Change in §3.9 [basic.types] paragraph 5:

A class that has been declared but not defined, an enumeration type in certain contexts (7.2), or an array of unknown size or of incomplete element type, is an incompletely-defined object type. Incompletely-defined object types ~~and the void types~~ are incomplete types (3.9.1). Objects shall not be defined to have an incomplete type.

Change in §3.9 [basic.types] paragraph 8:

An object type is a (possibly cv-qualified) type that is not a function type; ~~and~~ not a reference type; ~~and not a void type~~.

Change in §3.9 [basic.types] paragraph 9:

Arithmetic types (3.9.1), enumeration types, ~~void types~~, pointer types, pointer to member types (3.9.2), `std::nullptr_t`, and cv-qualified versions of these types (3.9.3) are collectively called *scalar types*.

Remove §3.9 [basic.types] paragraph 10.1 (regarding what is a literal type, remove redundancy now that `void` is a scalar type):

~~— possibly cv-qualified void; or~~

Change in §3.9.1 [basic.fundamental] paragraph 9:

The void type has ~~an empty set~~ ~~a unit set~~ of values. ~~The void type is an incomplete type that cannot be completed. It is used as the return type for functions that do not return a value.~~ Any expression can be explicitly converted to type cv `void` (5.4). ~~An expression of type void shall be used only as an expression statement (6.2), as an operand of a comma expression (5.19), as a second or third operand of `?:` (5.16), as the operand of `typeid`, `noexcept`, or `decltype`, as the expression in a return statement (6.6.3) for a function with the return type `void`, or as the operand of an explicit conversion to type cv `void`.~~

Change in §3.9.2 [basic.compound] paragraph 1.2 (regarding how compound types can be constructed, remove redundancy now that `void` is an object type):

~~— functions~~, which have parameters of given types and return ~~void or~~ references or objects of a given type, 8.3.5;

Change in §3.9.2 [basic.compound] paragraph 1.3 (regarding how compound types can be constructed, remove redundancy now that `void` is an object type):

~~— pointers~~ to ~~void or~~ objects or functions (including static members of classes) of a given type, 8.3.1;

Change in §3.9.2 [basic.compound] paragraph 3:

The type of a ~~pointer to void or~~ a pointer to an object type is called an *object pointer type*. ~~[Note: A pointer to void does not have a pointer-to-object type, however, because void is not an object type. — end note]...~~

Change in §3.9.3 [basic.type.qualifier] paragraph 1 (remove redundancy now that `void` is an object type):

A type mentioned in 3.9.1 and 3.9.2 is a *cv-unqualified type*. Each type which is a *cv-unqualified* complete or incomplete object type ~~or is void (3.9)~~ has three corresponding cv-qualified versions of its type: a *const-qualified* version, a *volatile-qualified* version, and a *const-volatile-qualified* version. The term object type (1.8) includes the cv-qualifiers specified in the decl-specifier-seq (7.1), declarator (Clause 8), type-id (8.1), or newtype-id (5.3.4) when the object is created.

Change in §3.10 [basic.lval] paragraph 4 (remove redundancy now that void is complete type):

Unless otherwise indicated (5.2.2), prvalues shall always have complete types ~~or the void type~~; in addition to these types, glvalues can also have incomplete types. [*Note*: class and array prvalues can have cv-qualified types; other prvalues always have cv-unqualified types. See Clause 5. — *end note*]

Change in §5.2.2 [expr.call] paragraph 3 (remove redundancy now that void is an object type):

If the *postfix-expression* designates a destructor (12.4), the type of the function call expression is void; otherwise, the type of the function call expression is the return type of the statically chosen function (i.e., ignoring the virtual keyword), even if the type of the function actually called is different. This return type shall be an object type; ~~or a reference type or cv-void~~.

Change in §5.2.3 [expr.type.conv] paragraph 2 (initialization now covered by value-initializing since void is an object type):

The expression T(), where T is a *simple-type-specifier* or *typename-specifier* for a non-array complete object type ~~or the (possibly cv-qualified) void type~~, creates a prvalue of the specified type, whose value is that produced by value-initializing (8.5) an object of type T; ~~no initialization is done for the void() case~~. [*Note*: if T is a non-class type that is cv-qualified, the cv-qualifiers are discarded when determining the type of the resulting prvalue (Clause 5). — *end note*]

Change in §5.3.1 [expr.unary.op] paragraph 1 (void is no longer incomplete):

The unary * operator performs indirection: the expression to which it is applied shall be a pointer to an object type, or a pointer to a function type and the result is an lvalue referring to the object or function to which the expression points. If the type of the expression is “pointer to T”, the type of the result is “T”. [*Note*: indirection through a pointer to an incomplete type ~~(other than cv-void)~~ is valid. The lvalue thus obtained can be used in limited ways (to initialize a reference, for example); this lvalue must not be converted to a prvalue, see 4.1. — *end note*]

Remove §5.3.5 [expr.delete] footnote 80:

~~This implies that an object cannot be deleted using a pointer of type void* because void is not an object type~~

Change in §5.9 [expr.rel] paragraph 1:

... The operands shall have arithmetic, enumeration, void, or pointer type. The operators < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to) all yield false or true. The type of the result is bool.

Change in §5.10 [expr.eq] paragraph 1:

The == (equal to) and the != (not equal to) operators group left-to-right. The operands shall have arithmetic, enumeration, void, pointer, or pointer to member type, or type std::nullptr_t. The operators == and != both yield true or false, i.e., a result of type bool. In each case below, the operands shall have the same type after the specified conversions have been applied.

Add new paragraph to §5.10 [expr.eq]:

Two operands of type void compare equal.

Change in §5.16 [expr.cond] paragraph 2 (regarding the conditional operator):

If either the second or the third operand ~~has type void~~ is a (possibly parenthesized) throw-expression (5.17), one of the following shall hold:

(2.1) — The second or the third operand (but not both) is a (possibly parenthesized) throw-expression ~~(5.17)~~; the result is of the type and value category of the other. The conditional-expression is a bit-field if that operand is a bit-field.

(2.2) — Both the second and the third operands ~~have type void~~ are a (possibly parenthesized) throw-expression; the result is of type void and is a prvalue. [*Note*: This includes the case where both operands are throw-expressions. — *end note*]

Change in §6.6.3 [stmt.return] paragraph 2:

The expression or braced-init-list of a return statement is called its operand. A return statement with no operand shall be used only in a function whose return type is cv void, a constructor (12.1), or a destructor (12.4). ~~A return statement with an operand of type void shall be used only in a function whose return type is cv void. A return statement with any other operand shall be used only in a function whose return type is not cv void; the~~ A return statement with ~~any other~~ an operand initializes the object or reference to be returned by *copy-initialization* (8.5) from the operand. [*Note*: A return statement can involve the construction and copy or move of a temporary object (12.2). A copy or move operation associated with a return statement may be elided or considered as an rvalue for the purpose of overload resolution in selecting a constructor (12.8). — *end note*] [*Example*:

```
std::pair<std::string,int> f(const char* p, int x) {  
    return {p,x};  
}
```

— end example]

Change in §8.3.2 [dcl.ref] paragraph 1. Remove the final statement starting after the end of the last note:

~~A declarator that specifies the type “reference to cv void” is ill formed.~~

Change in §8.3.3 [dcl.mptr] paragraph 3:

A pointer to member shall not point to a static member of a class (9.4); or a member with reference type; ~~or “cv void”.~~

[*Note*: See also 5.3 and 5.5. The type “pointer to member” is distinct from the type “pointer”, that is, a pointer to member is declared only by the pointer to member declarator syntax, and never by the pointer declarator syntax. There is no “reference-to-member” type in C++. — *end note*]

Change in §8.3.4 [dcl.array] paragraph 1:

... T is called the array element type; this type shall not be a reference type, ~~the (possibly cv-qualified) type void~~; a function type or an abstract class type. ...

Change in §8.3.4 [dcl.array] paragraph 2:

An array can be constructed from one of the fundamental types (~~except void~~), from a pointer, from a pointer to member, from a class, from an enumeration type, or from another array.

Change in §8.3.5 [dcl.fct] paragraph 4:

The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [*Note*: the *parameter-declaration-clause* is used to convert the arguments specified on the function call; see 5.2.2. — *end note*] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type void is equivalent to an empty parameter list [*Note*: This usage of void is deprecated. — *end note*]. ~~Except for this special case, a parameter shall not have type cv void. This is the only case for which a parameter of type cv void has special meaning.~~ If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack (14.5.3), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where “...” is not part of an abstract-declarator, “...” is synonymous with “...”. [*Example*: the declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");  
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a const char* — end example] [*Note*: The standard header <stdarg.h> contains a mechanism for accessing arguments passed using the ellipsis (see 5.2.2 and 18.10). — *end note*]

Change in §9.4.2 [class.static.data] paragraph 2 (void is no longer incomplete):

The declaration of a static data member in its class definition is not a definition and may be of an incomplete type ~~other than cv-qualified void~~....

Change in §13.6 [over.built] paragraph 15 (regarding comparison operator declarations):

For every T, where T is an enumeration type, **void** or a pointer type, there exist candidate operator functions of the form

```
bool operator<(T , T );
bool operator>(T , T );
bool operator<=(T , T );
bool operator>=(T , T );
bool operator==(T , T );
bool operator!=(T , T );
```

Change in §14.1 [temp.param] paragraph 7:

A non-type template-parameter shall not be declared to have floating point, **or** class, **or void** type....

Change in §14.8.2 [temp.deduct] paragraph 8.2 (details regarding when type deduction can fail):

— Attempting to create an array with an element type that is **void**, a function type, a reference type, or an abstract class type, or attempting to create an array with a size that is zero or negative....

Remove §14.8.2 [temp.deduct] paragraph 8.6 (details regarding when type deduction can fail):

~~— Attempting to create a reference to void~~

Change in §14.8.2 [temp.deduct] paragraph 8.10 (details regarding when type deduction can fail):

— Attempting to create a function type in which **a parameter has a type of void, or in which** the return type is a function type or array type.

Change in §15.1 [except.throw] paragraph 3 (void is no longer incomplete):

Throwing an exception copy-initializes (8.5, 12.8) a temporary object, called the *exception object*. The temporary is an lvalue and is used to initialize the variable declared in the matching *handler* (15.3). If the type of the exception object would be an incomplete type or a pointer to an incomplete type **other than (possibly cv-qualified) void** the program is ill-formed.

Change in §15.3 [except.handle] paragraph 1 (void is no longer incomplete):

... The *exception-declaration* in a *handler* describes the type(s) of exceptions that can cause that *handler* to be entered. The *exception-declaration* shall not denote an incomplete type, an abstract class type, or an rvalue reference type. The *exception-declaration* shall not denote a pointer or reference to an incomplete type, **other than void*, const void*, volatile void*, or const volatile void***.

Change in §15.4 [except.handle] paragraph 2 (void is no longer incomplete):

A type denoted in a *dynamic-exception-specification* shall not denote an incomplete type or an rvalue reference type. A type denoted in a *dynamic-exception-specification* shall not denote a pointer or reference to an incomplete type; **other than “pointer to cv void”**. A type cv T, “array of T”, or “function returning T” denoted in a dynamic-exception-specification is adjusted to type T, “pointer to T”, or “pointer to function returning T”, respectively. A dynamic-exception-specification denotes an exception specification that is the set of adjusted types specified thereby.

Change in §20.10.4.3 Type properties [meta.unary.prop]:

Remove explicit mentions of void other than in is_void (*complicated to list, spanning across multiple tables*).

Acknowledgments

Though a change to the language such as this has been discussed informally by several in the past, particularly by Sean Parent, the direct motivation for this proposal was a discussion³ in the std-proposals group (a thread that the author of this proposal did not start). Many suggestions and alternatives mentioned in this proposal come from those who took part in that discussion.

In addition to those mentioned above, Richard Smith provided support and early feedback during the writing of this proposal. I also hijacked his formatting for use in this document.

References

[1] James C. Dehnert and Alexander Stepanov: "Fundamentals of Generic Programming"
<http://www.stepanovpapers.com/DeSt98.pdf>

[2] Vicente J. Botet Escriba and Pierre Talbot: "A proposal to add a utility class to represent expected monad" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4109.pdf>

[3] *Various Participants* "Allow values of void" Discussion <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/05prNzycvYU>